Rashad Cader

IT22331472

Undergrad Thesis Checking

Document Details

Submission ID

trn:oid:::3618:136603304

Submission Date

Apr 26, 2026, 11:27 AM GMT+5:30

Download Date

Apr 26, 2026, 11:35 AM GMT+5:30

File Name

IT22331472.docx

File Size

998.5 KB

39 Pages

8,298 Words

49,790 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

23 Not Cited or Quoted 5%
Matches with neither in-text citation nor quotation marks

5 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
2% Publications
4% Submitted works (Student Papers)

Match Groups

- **23** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **5** Missing Quotations 1%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
2% Publications
4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Sri Lanka Institute of Information Technology on 2021-10-09	2%
2	Student papers	National Institute of Business Management Sri Lanka on 2023-08-25	<1%
3	Student papers	Sri Lanka Institute of Information Technology on 2021-10-07	<1%
4	Internet	uwindsor.scholaris.ca	<1%
5	Student papers	University of Essex on 2026-04-24	<1%
6	Internet	www.diva-portal.org	<1%
7	Internet	dl.lib.uom.lk	<1%
8	Internet	research.library.mun.ca	<1%
9	Student papers	De LaSalle - College of Saint Benilde on 2026-04-17	<1%
10	Internet	core.ac.uk	<1%

11 Internet

dr.ur.ac.rw <1%

12 Internet

doktori.bibl.u-szeged.hu <1%

13 Internet

slidelegend.com <1%

14 Internet

www.alumni.uz.ac.zw <1%

15 Student papers

Monash University on 2021-04-08 <1%

16 Student papers

Wawasan Open University on 2025-03-21 <1%

17 Student papers

Auckland University of Technology on 2011-05-17 <1%

18 Internet

www.iied.org <1%

19 Publication

Joy Ong, Low Sui Pheng. "Waste Reduction in Precast Construction", Springer Scie... <1%

20 Publication

Leah C. Hall. "Risk Perceptions in RPA Implementation: Exploring Internal Auditor... <1%

21 Publication

Rohit Punnoose, Supriya Kumar De. "Phase-wise migration of multiple legacy app... <1%

22 Internet

sautarusha.ac.tz <1%

23 Internet

trap.ncirl.ie <1%

24 Internet

unsworks.unsw.edu.au <1%

LEVERAGING AGENTIC AI MIDDLEWARE FOR INTELLIGENT INTERACTION AND MODERNIZATION OF LEGACY ENTERPRISE APPLICATIONS

S.M.A.C.M Rashad

(IT22331472)

2

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

May 2026

LEVERAGING AGENTIC AI MIDDLEWARE FOR INTELLIGENT INTERACTION AND MODERNIZATION OF LEGACY ENTERPRISE APPLICATIONS

S.M.A.C.M Rashad

(IT22331472)

1

Dissertation submitted in partial fulfillment of the requirements for the

Bachelor of Science in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

May 2026

DECLARATION

We declare that this is our own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute our dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
S M A C M Rashad	IT22331472	

15

Signature of the Supervisor: ……

(Mr. Vishan Jayasinghearachchi)

Signature of the Co-Supervisor: ……

(Dr. Dharshana Kasthurirathna)

Date: 20/08/2025

Abstract

Legacy enterprise applications continue to support critical operations in healthcare, finance, government, and other high-continuity sectors. Many of these systems remain operationally valuable because they contain stable domain logic and mature workflows, but they are difficult to integrate with modern platforms because they expose only graphical user interfaces and do not provide APIs. Full replacement, re-engineering, or database-level integration may be costly, risky, or impossible when source code is unavailable. Robotic Process Automation (RPA) offers a non-intrusive alternative, but conventional RPA scripts often fail when screen positions change, controls lose focus, a virtual machine is moved, or a legacy control behaves differently from modern UI components.

This individual research component focuses on the RPA Bot Layer for legacy system modernization. The component was designed and implemented to interact with a Visual Basic 6 hospital management application running inside a Windows XP virtual machine. The bot layer operates only through the visible user interface and does not require changes to the legacy application source code or database. The implementation combines PyAutoGUI for general GUI control and screenshots, PyDirectInput for more reliable keyboard dispatch, AutoHotkey for low-level fallback typing, OpenCV for visual control and screen-state detection, Tesseract OCR for text extraction, relative-coordinate calibration for VM movement tolerance, and structured before/after JSON logs with VM screenshots for auditability.

The implemented workflows include Add Patient, Read Patient Details, Treatment Entry, and Delete/Treatment operations. The Add Patient workflow uses a vision-first bot that detects form controls from live screenshots and falls back to calibrated coordinate automation when needed. Additional add-patient agents provide coordinate-driven and AutoHotkey-driven alternatives. The treatment workflows use dialog detection, field-level typing verification, runtime error handling, and screen recovery to reduce unsafe execution. Evaluation from the draft study reports a field-level input failure rate of 1.9%, OpenCV-based field location accuracy of 96.7%, and OCR field-level accuracy of 94.1%. Local structured logs in the current repository further show six successful Add Patient runs out of six recorded attempts, five successful Add Treatment runs out of eight, and one successful Delete Treatment run out of one. These results show that a hybrid, vision-supported RPA Bot Layer can provide a practical transition mechanism for modernizing GUI-only legacy systems, while also identifying areas that require further hardening before production deployment.

Keywords

Legacy systems, Robotic Process Automation, GUI automation, Windows XP, Visual Basic 6, PyAutoGUI, PyDirectInput, AutoHotkey, OpenCV, Tesseract OCR, screen-state detection, low-code modernization.

22

DEDICATION

This dissertation is dedicated to my family, supervisors, and all those whose unwavering support and encouragement made this work possible.

8

ACKNOWLEDGEMENTS

First and foremost, I would like to express my sincere gratitude to my supervisors, Mr. Vishan Jayasinghearachchi and Dr. Dharshana Kasthurirathna, of the Department of Computer Science and Software Engineering at the Sri Lanka Institute of Information Technology. Their unwavering guidance, timely feedback, and technical expertise were indispensable at every stage of this research. Their encouragement during the most challenging phases of the project particularly during the integration of the multi-agent orchestration layer with the legacy VM environment gave me the confidence and direction needed to persevere.

I also extend my heartfelt appreciation to the co-supervisors and panel members who provided constructive critique during the proposal and progress reviews. Their suggestions shaped the experimental evaluation methodology and strengthened the scientific rigour of the research findings. Special thanks go to the industry professionals from the healthcare IT sector in Sri Lanka who shared their operational experiences with legacy hospital management systems, providing invaluable practical context that grounded the research problem in real-world constraints.

I am deeply grateful to the technical staff of the SLIIT Computing Infrastructure team for their assistance in configuring the virtualized evaluation environment, including the Oracle VirtualBox instance running Windows XP SP3 used for evaluation. Access to this infrastructure was essential for the empirical component of this research.

Finally, I thank my family and friends, whose moral support throughout the long hours of development, testing, and writing sustained me through this journey.

TABLE OF CONTENT

DECLARATION iii
Abstract iv
Keywords iv
DEDICATION v
ACKNOWLEDGEMENTS vi
LIST OF FIGURES ix
LIST OF ABBREVIATIONS xi
1. INTRODUCTION 1
1.1 Literature Review 2
1.1.1 Legacy Systems and Modernization Pressure 2
1.1.2 Conventional Modernization Strategies 2
1.1.3 RPA for Legacy Integration 2
1.1.4 Computer Vision and OCR for GUI Understanding 2
1.1.5 GUI State and Self-Healing Automation 3
1.2. Research Gap 3
1.3 Research Problem 4
1.4 Research Objectives 5
1.4.1 Main Objective 5
1.4.2 Specific Objectives 5
2. METHODOLOGY 6
2.1 Research Approach 6
2.2 Target Environment 7
2.3 Relative-Coordinate Calibration 7
2.4 Vision-Based Control Detection 8
2.5 Hybrid Input Injection 8
2.6 Screen-State Detection and Recovery 9
2.7 OCR-Based Extraction 9
2.8 Structured Logging and Observability 9
3. SYSTEM DESIGN AND IMPLEMENTATION 11
3.1 Overall Component Architecture 11
3.2 VM Window Handling 11
3.3. Add Patient Workflow 11
3.4 Read Patient Details Workflow 13
3.5 Treatment Entry Workflow 13
3.6 Delete/Treatment Workflow 15
3.7 Screen-State Recovery/ Error Handling 15

5 4. TESTING AND EVALUATION 16
4.1 Evaluation Goals 16
4.2. Test Cases 16
4.3 Metrics 16
4.4. Log-Based Evaluation 17
12 4.5 Gantt Chart 18
5. RESULTS AND DISCUSSION 19
5.1 Local Structured Log Results 19
5.2 Add Patient Agent Comparison 20
5.3. Component Accuracy Results 20
5.4 Discussion 21
17 6. LIMITATIONS 22
7. FUTURE WORK 23
8. CONCLUSION 24
REFERENCES 25
APPENDICES 26
Appendix A: Suggested Figures for Final Word Submission 26
Appendix B: Implementation Evidence 26
Appendix C: Commercialization and Deployment Outlook 27
Appendix D: Evidence-Based Limitations Summary 27

13

LIST OF FIGURES

Figure 1: High- Level System Architecture Diagram 6
Figure 2: RPA Navigation 7
Figure 3: RPA add patient workflow 12
Figure 4: RPA read patient details 13
Figure 5: RPA add treatment details 14
Figure 6: RPA remove patient 15
Figure 7: Gantt Chart 18

LIST OF TABLES

Table 1: RPA Bot Layer Technology Stack 8
Table 2: Main Implemented Add Patient Workflow Strategies..... 12
Table 3: Local Structured Log Evaluation Results 19
Table 4: Add Patient Agent Comparison 20
Table 5: Limitations and Future Improvements..... 27

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CRUD	Create, Read, Update, Delete
GITT	GUI Interaction Task Translator
GUI	Graphical User Interface
JSON	JavaScript Object Notation
LAMA	Legacy Application Modernization Architecture
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition
RPA	Robotic Process Automation
SCRUM	Agile Software Development Framework
SLIIT	Sri Lanka Institute of Information Technology
SME	Small and Medium Enterprise
SOACRS	Self-Optimizing Agent Coordination and Response System
UI	User Interface
VM	Virtual Machine
WebSocket	Web Communication Protocol (RFC 6455)

1. INTRODUCTION

Legacy software systems remain deeply embedded in many organizations because they continue to perform essential business operations reliably. In healthcare, finance, public administration, transportation, and similar sectors, legacy systems often contain decades of validated business rules, user practices, and operational data. Although these applications may be old, replacing them is not always practical. A complete rewrite or migration can introduce downtime, data loss, training cost, and the risk of losing rules that are encoded only in the behavior of the existing application [1], [2].

The modernization difficulty becomes greater when the legacy system exposes only a graphical user interface (GUI). Many older applications were built before modern API-first architecture became common. A hospital system developed using Visual Basic 6, for example, may allow users to add patients, search records, enter treatments, and delete treatment data through Windows forms, but may not provide REST APIs, database documentation, or service interfaces. This means that modern platforms cannot easily call the system programmatically. Even simple goals such as web access, mobile integration, automated reporting, or workflow orchestration become difficult when the only stable interface is the screen itself.

Traditional modernization approaches such as re-hosting, re-platforming, and re-engineering attempt to move or rebuild the legacy system using newer infrastructure. These approaches are valuable in some situations, but they usually require significant access to the application internals and may introduce high cost and operational risk [1], [3]. For organizations that cannot interrupt daily operations, a less invasive transition layer is often more realistic.

Robotic Process Automation (RPA) provides such a transition layer by automating the same actions that a human user performs: clicking buttons, typing values, reading screen content, and navigating forms [4], [5]. Because RPA can operate through the GUI, it is attractive for systems where the source code or database cannot be changed. However, conventional RPA is often fragile. Scripts that depend on absolute coordinates can fail if the window moves. Selector-based automation can fail when older Win32 controls expose weak or inconsistent metadata. Bots can also fail when a virtual machine loses focus, a screen loads slowly, or a modal dialog appears unexpectedly.

This research focuses on the RPA Bot Layer within the broader project "Create a Low-Code RPA Connector Layer for Legacy Systems" under Project ID 25-26J-455. The broader project considers agentic middleware, task translation, simulation, and modern API exposure. This individual component is narrower and more execution-focused: it is responsible for reliably operating the legacy GUI itself. In the implemented prototype, the target system is a Visual Basic 6 hospital application running inside a Windows XP virtual machine. The bot layer uses a combination of relative-coordinate calibration, computer vision, OCR, multiple input libraries, screen-state detection, and structured logging to automate key hospital workflows without modifying the legacy software.

The contribution of this component is practical as well as research-oriented. It demonstrates that GUI-only modernization does not need to depend only on brittle fixed-coordinate scripts. By combining visual perception, calibrated fallback, hybrid input injection, and auditable execution logs, the RPA Bot Layer can serve as a dependable bridge between a legacy desktop system and higher-level orchestration services.

24

1.1 Literature Review

1.1.1 Legacy Systems and Modernization Pressure

21

Legacy systems are commonly defined as older information systems that remain critical to organizational operations but are difficult to maintain, integrate, or evolve [1]. Bennett notes that many legacy systems continue to exist because they are successful: they support important business processes and therefore cannot simply be discarded [2]. This creates a modernization paradox. The system is valuable enough to keep, but old enough to limit future development.

In critical industries, this problem is especially serious. Healthcare providers, banks, government departments, and transport organizations often rely on systems built using older technologies such as COBOL, Visual Basic, PowerBuilder, or mainframe platforms. These applications may be stable in daily use, but they can restrict interoperability, security upgrades, analytics, and digital service delivery. Charette highlights that legacy IT remains a hidden but major operational burden for large organizations [3].

1.1.2 Conventional Modernization Strategies

Common modernization strategies include re-hosting, re-platforming, and re-engineering. Re-hosting moves an existing workload to newer infrastructure with minimal code changes. Re-platforming makes limited technical changes to run the application on a newer managed environment. Re-engineering or rewriting attempts to redesign the application using modern architecture [1], [3].

These strategies can be effective, but they are not always appropriate for fragile or poorly documented systems. Re-hosting may preserve old technical debt. Re-platforming may still leave the organization dependent on old architecture. Re-engineering can provide long-term benefits, but it requires high cost, skilled personnel, and deep knowledge of the existing application. In systems where the original source code is missing or poorly understood, code-level modernization may not be feasible.

1.1.3 RPA for Legacy Integration

RPA emerged as a non-intrusive method for automating repetitive business processes by simulating user actions at the interface level [4], [5]. In a legacy modernization context, RPA is useful because it can interact with an existing application through the same GUI used by human operators. It does not require the legacy source code to be changed and can be deployed incrementally.

However, RPA has a known maintenance problem. Traditional bots are often built using fixed coordinates, image snippets, or UI selectors. If a button moves, a window is resized, or a screen changes slightly, the automation can break. This issue is commonly described as UI drift. Large RPA deployments therefore require continuous monitoring and maintenance, which reduces the long-term benefit of automation [5], [6].

1.1.4 Computer Vision and OCR for GUI Understanding

Computer vision and OCR techniques can reduce the fragility of GUI automation by allowing bots to reason about what is visible on the screen. Instead of clicking a fixed pixel location, a bot can detect the approximate position of an input box or button from the screenshot. Visual

methods are particularly valuable for older applications that do not expose useful accessibility trees or modern UI automation identifiers.

Research on GUI understanding has shown that screen images can be transformed into structured interface representations. Beltramelli's pix2code demonstrated that GUI screenshots can be interpreted for interface generation tasks [7]. The RICO dataset supported large-scale research on UI structure and mobile interface understanding [8]. More recent work such as OmniParser and UI-TARS explores vision-grounded interaction with graphical interfaces [9], [10]. These studies show that visual understanding can support automation in dynamic interfaces, although many research prototypes do not directly address the reliability needs of enterprise legacy systems.

OCR is also important because many legacy systems expose data only as rendered text. Tesseract OCR provides a practical foundation for extracting text from images [11]. In GUI automation, OCR can be used to read patient IDs, table rows, error messages, labels, and confirmation dialogs when no backend API is available. OCR quality depends on preprocessing, segmentation, character constraints, and screen quality, so an RPA system must treat OCR as a probabilistic support mechanism rather than a perfect data source.

1.1.5 GUI State and Self-Healing Automation

Reliable GUI automation must consider application state. GUI testing research has long shown that applications can be modeled through events and screen transitions [12]. This is relevant to RPA because a click is not safe merely because a coordinate exists. The bot must also know whether the current screen is the dashboard, patient list, treatment history, add-patient form, or an error dialog. If an action is executed on the wrong screen, the result may be data corruption or workflow failure.

Adaptive RPA research and agentic AI research further motivate recovery-oriented automation. Self-correcting systems can retry actions, re-plan, and use feedback when an action fails [13], [14]. In the context of this project, the RPA Bot Layer does not implement a fully autonomous agent, but it applies the same principle at the execution level by using screen-state checks, visual detection, fallback input strategies, and before/after evidence.

1.2. Research Gap

The proposal identified a broad gap in the absence of an end-to-end AI-powered connector that can transform GUI-bound legacy workflows into secure programmable services. Existing modernization strategies often require source-code access, database migration, or major infrastructure change. Traditional RPA tools can automate existing GUIs, but they are brittle when the interface changes or the execution environment behaves unpredictably. Computer-vision and OCR research improves screen understanding, but many approaches stop at interface parsing rather than verified operational execution.

For this individual component, the research gap is more specific. There is a need for an execution-focused RPA Bot Layer that can operate a real GUI-only Win32 legacy application inside a virtual machine while remaining robust enough for repeated workflow automation. Such a layer must satisfy several practical conditions:

- It must be non-intrusive and avoid modification of the legacy Visual Basic 6 application.

- It must work through a Windows XP virtual machine, where focus and keyboard routing can be unreliable.
- It must reduce dependence on absolute coordinates by using relative calibration and visual detection.
- It must support multiple input strategies because PyAutoGUI, PyDirectInput, and AutoHotkey each behave differently with legacy controls.
- It must read data back from the screen using OCR when no API or database connection is available.
- It must detect the current screen before performing sensitive actions.
- It must produce execution evidence through structured logs and screenshots.

The proposal also described dynamic GUI simulation and REST/GraphQL API exposure. These remain important for the wider modernization architecture, but the final implemented contribution of this report is the bot execution layer that makes those higher-level services possible. Without a reliable RPA Bot Layer, simulation, orchestration, and API exposure cannot safely interact with the legacy system.

16

1.3 Research Problem

The research problem addressed in this report is:

How can an RPA Bot Layer reliably interact with a GUI-only Visual Basic 6 hospital application running inside a Windows XP virtual machine, using only interface-level input and screen observation, while remaining robust to focus problems, UI drift, VM repositioning, rendering delays, OCR limitations, and legacy control behavior, without modifying the original legacy software?

This problem can be divided into the following sub-problems:

- How can the bot locate fields and buttons without depending only on fixed screen coordinates?
- How can the bot keep working when a virtual machine window is moved, resized, restored, or maximized?
- How can the bot enter text reliably when modern input libraries fail to send keystrokes into a legacy VM?
- How can the bot confirm that it is on the correct screen before performing actions such as adding a patient or deleting treatment data?
- How can the system read patient records from the rendered GUI when APIs and database hooks are unavailable?
- How can each automation attempt be recorded with enough evidence for debugging, evaluation, and audit?

9

1.4 Research Objectives

1.4.1 Main Objective

The main objective is to design and implement a reliable RPA Bot Layer for legacy GUI automation that enables safe, non-intrusive interaction with a virtualized Visual Basic 6 hospital application using computer vision, OCR, hybrid input injection, relative-coordinate calibration, screen-state detection, and structured execution logging.

14

1.4.2 Specific Objectives

The specific objectives are:

1. To implement relative-coordinate calibration so navigation and workflow actions remain stable when the Windows XP virtual machine is moved or displayed at different host positions.
2. To integrate OpenCV-based screen and control detection for workflows where live visual detection is more reliable than static coordinates.
3. To implement a hybrid input strategy using PyAutoGUI, PyDirectInput, and AutoHotkey to improve typing and clicking reliability inside a virtualized Win32 environment.
4. To implement OCR-based reading of patient details and screen text using Tesseract-supported pipelines and preprocessing methods.
5. To implement screen-state detection and recovery for dashboard, patient list, treatment history, add-patient form, and unknown states.
6. To support Add Patient, Read Patient Details, Treatment Entry, and Delete/Treatment workflows through reusable bot primitives.
7. To record structured before/after JSON logs and VM screenshots for each task attempt.
8. To evaluate the bot layer using accuracy, success rate, failure rate, duration, and observability evidence.

18

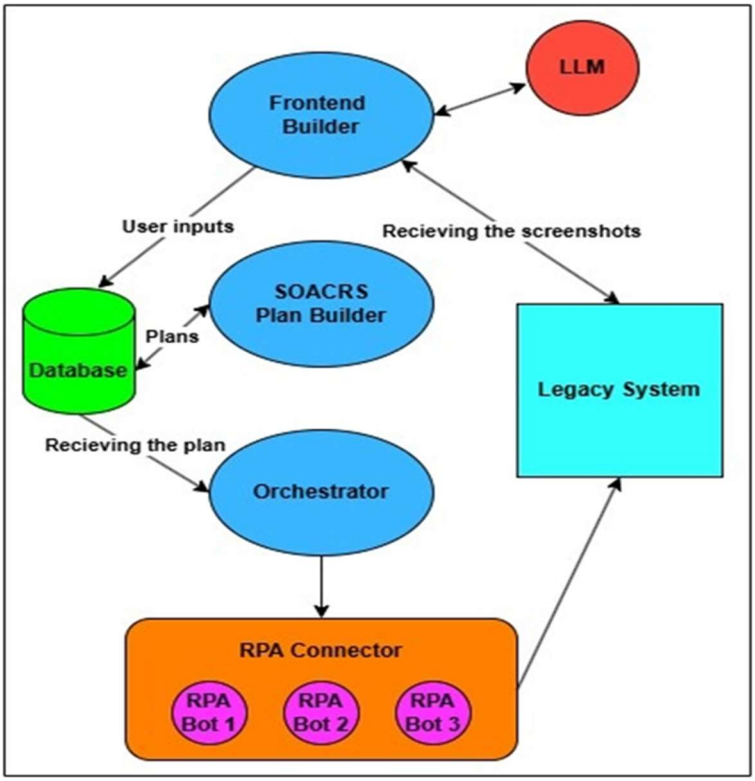
2. METHODOLOGY

2.1 Research Approach

The methodology follows a design-and-build approach. Instead of modifying the legacy application, the solution wraps it externally through GUI automation. The bot layer is treated as a non-intrusive connector between higher-level workflow requests and the legacy interface. The target application is a Visual Basic 6 hospital management system running in a Windows XP virtual machine named "WinXP for VB6". The bot interacts with the system through mouse movement, clicks, keyboard input, screenshots, computer vision, and OCR.

The implemented methodology is divided into six parts:

- VM focus and screenshot capture.
- Relative-coordinate calibration.
- Vision-based control detection.
- Hybrid input injection and fallback.
- OCR-based extraction and screen-state detection.
- Structured logging and evaluation.



23

Figure 1: High- Level System Architecture Diagram

2.2 Target Environment

The target environment was selected to reflect a realistic legacy modernization scenario. The hospital system is a desktop Visual Basic 6 application. It is not assumed to expose modern APIs. The application runs inside a Windows XP virtual machine, which introduces practical automation challenges such as host-to-guest focus routing, older Win32 controls, modal dialogs, and inconsistent typing behavior.

The use of a virtual machine also makes calibration important. If absolute screen coordinates are stored directly, a click that works in one VM position may fail after the VM window is moved. Therefore, the bot layer uses relative coordinates wherever possible.

2.3 Relative-Coordinate Calibration

Relative-coordinate calibration stores target points as a ratio of the active VM window instead of as fixed host screen pixels. If the VM window has top-left coordinate `(x_vm, y_vm)`, width `w_vm`, and height `h_vm`, and the target point is at absolute coordinate `(x, y)`, the normalized coordinates are:

```
r_x = (x - x_vm) / w_vm
```

```
r_y = (y - y_vm) / h_vm
```

During execution, the absolute target position is reconstructed as:

```
x_abs = x_vm + (r_x * w_vm)
```

```
y_abs = y_vm + (r_y * h_vm)
```

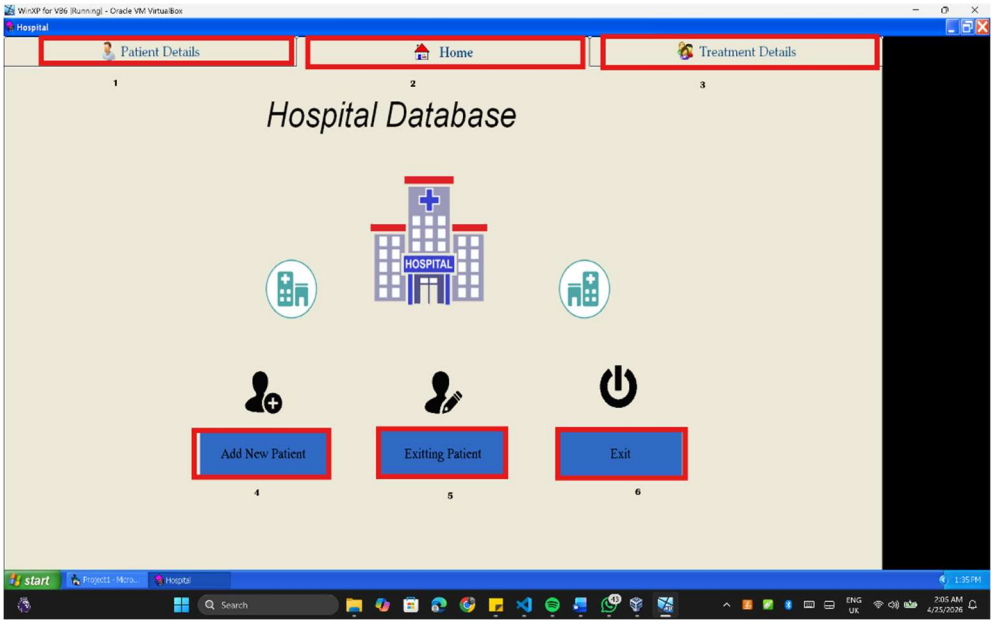


Figure 2: RPA Navigation

This method is used in dashboard navigation, patient details navigation, treatment workflow controls, and screen-recovery helpers. It allows the bot to click the same logical area of the VM even after the window is repositioned or maximized. Some older add-patient fallback paths still use stored pixel offsets, which is treated as a limitation and future improvement.

2.4 Vision-Based Control Detection

For the Add Patient and Treatment Entry workflows, the bot captures the current VM screen and processes it using OpenCV. The main steps include:

- Capture the VM window region using PyAutoGUI.
- Convert the screenshot to grayscale or HSV as required.
- Apply thresholding to isolate bright form controls or dark dialog regions.
- Extract contours using OpenCV.
- Filter bounding boxes by size, position, and area.
- Remove near-duplicate detections.
- Sort detected controls spatially.
- Partition controls into editable fields and action buttons.

In the Add Patient workflow, this process identifies the patient form fields and the Add/Return buttons. In the Treatment Entry workflow, the process identifies the Fill Details dialog, editable fields, and action buttons. The purpose is not to create a full semantic model of the UI, but to make the bot more resilient than a purely coordinate-based script.

2.5 Hybrid Input Injection

No single input method was reliable for all legacy controls in the Windows XP virtual machine. The implementation therefore uses a hybrid strategy:

Table 1: RPA Bot Layer Technology Stack

Tool	Role in the Bot Layer
PyAutoGUI	General mouse movement, clicking, screenshots, hotkeys, and fallback typing
PyDirectInput	More reliable key press dispatch when PyAutoGUI keystrokes are missed by the VM
AutoHotkey	Low-level fallback typing for difficult dialogs, patient ID entry, and treatment fields
Pyperclip	Clipboard-based paste support for stable text insertion
OpenCV	Form, dialog, control, and state detection
Tesseract OCR	OCR-supported patient table and screen text extraction

The bot attempts the most convenient interaction path first and escalates when necessary. For example, the Add Patient Agent 1 workflow uses live visual detection first and falls back to calibrated coordinate entry if detection fails. The Treatment Entry workflow attempts AutoHotkey-based typing and falls back to clipboard-based entry when AutoHotkey is unavailable. This layered design improves resilience because it does not assume that one library will behave correctly across every legacy control.

2.6 Screen-State Detection and Recovery

The screen-state detection module classifies the current VM screen into one of the following states:

- Dashboard.
- Patient list.
- Treatment history.
- Add Patient form.
- Unknown.

The implementation uses color-distribution heuristics for major screens and visual control detection for the Add Patient form. For example, specific hue ranges are used to distinguish dashboard, patient list, and treatment history views. If the bot is not on the expected screen, recovery helpers click calibrated navigation buttons to return to a known state. This reduces the risk of executing valid clicks in the wrong context.

2.7 OCR-Based Extraction

OCR is used when the bot must read information from the legacy GUI. The patient details workflow captures a VM screenshot and extracts screen text. Additional OCR utilities use preprocessing steps such as denoising, contrast enhancement, adaptive thresholding, table detection, column segmentation, scaling, and character whitelists. For patient tables, field-specific constraints are applied for patient ID, age, date of birth, sex, patient name, and address.

The proposal described OCR as part of a broader screen-recognition pipeline. In the implemented component, OCR is used mainly for readback, patient details extraction, and dialog/button interpretation. Because OCR can misread long or visually compressed table values, the results are treated as useful evidence but not as perfect truth in high-risk workflows.

2.8 Structured Logging and Observability

Each task can be logged using a before/after pattern. At the start of a task, the bot generates a unique run ID, stores the input payload, records metadata, and optionally captures a VM screenshot. At the end of the task, it records success status, result data, error messages, metadata, the path to the before log, and an after screenshot.

This design provides:

- Evidence of what the VM looked like before and after execution.
- Payload traceability.
- Failure messages for debugging.
- Duration calculation from timestamps.
- A basis for success-rate and observability scoring.

This logging mechanism is important in healthcare-like environments because automation must be explainable and auditable, especially when patient data or treatment data is involved.

6

3. SYSTEM DESIGN AND IMPLEMENTATION

3.1 Overall Component Architecture

The RPA Bot Layer is positioned below higher-level orchestration and task translation services. Its role is to convert structured workflow intent into verified GUI actions. The layer accepts a payload such as patient data or treatment data, focuses the VM, detects or navigates to the required screen, performs the GUI actions, captures evidence, and returns success or failure to the caller.

Fig. 1 should show the RPA Bot Layer as the execution layer between modern orchestration/API services and the Windows XP Visual Basic 6 hospital application.

The component is organized into the following modules:

- VM window handling and focus control.
- Calibration and coordinate conversion.
- Add Patient automation agents.
- Patient Details OCR and reading.
- Treatment Entry automation.
- Delete/Treatment dialog automation.
- Screen-state detection and recovery.
- Before/after logging and log analysis.

3.2 VM Window Handling

The VM helper module locates the best visible VM window using the preferred title "WinXP for VB6 [Running] - Oracle VM VirtualBox" and the fallback title "WinXP for VB6". It restores, activates, optionally maximizes, and optionally clicks inside the VM window. A clamped screenshot region is used to avoid invalid coordinates when a window is partially outside the host display.

This implementation detail is important because many failures in GUI automation occur before the workflow even begins. If the wrong window is focused, text may be typed into the host operating system instead of the guest VM. The bot therefore treats VM focus as a first-class part of the execution process.

3.3. Add Patient Workflow

The Add Patient workflow is the main implemented patient-registration workflow. It fills a Visual Basic 6 form with the following fields:

- Patient name.
- Address.
- Date of birth.
- Age.
- Sex.

The sex field is normalized to a single character, usually "M" or "F", because the legacy form expects a compact value.

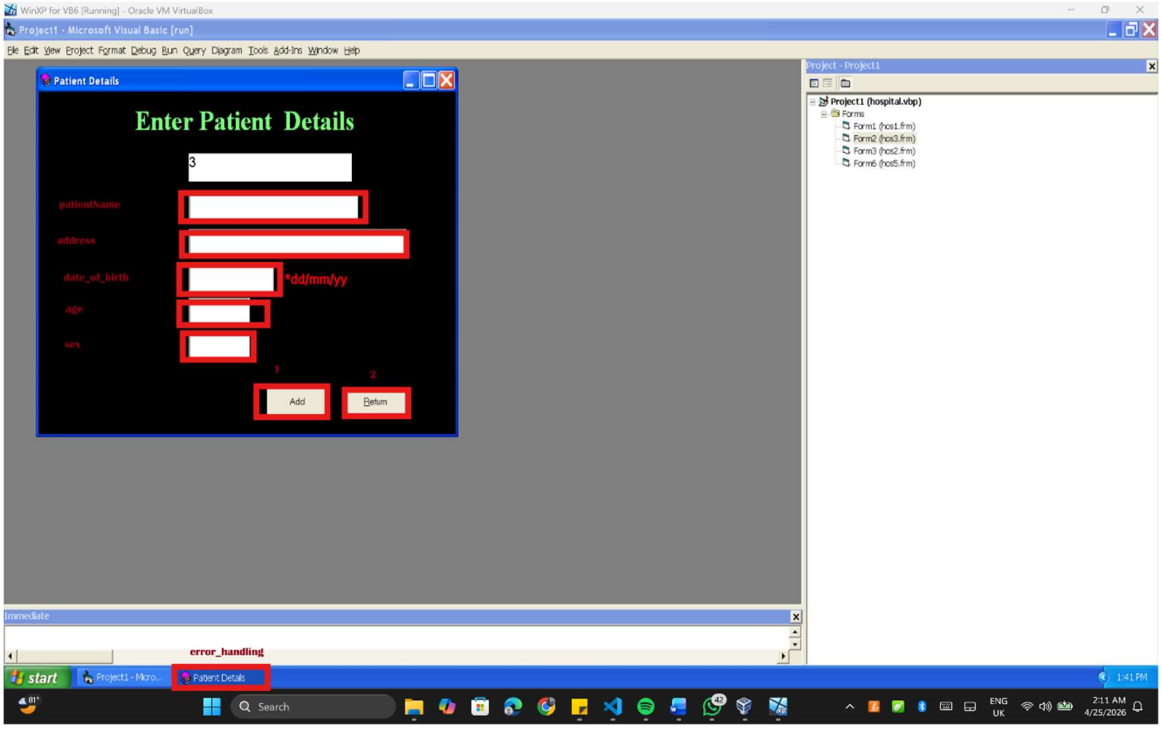


Figure 3: RPA add patient workflow

The current implementation provides three add-patient execution paths:

Table 2: Main Implemented Add Patient Workflow Strategies

Agent	Strategy	Implementation Role
Agent 1	Vision-first control detection with coordinate fallback	Main benchmarked Add Patient path
Agent 2	Calibrated coordinate entry using PyDirectInput and clipboard paste	Faster coordinate-driven alternative
Agent 3	AutoHotkey runner using an INI payload and calibrated coordinates	Low-level fallback route for difficult input routing

Agent 1 is the most complete path. It normalizes the incoming payload, starts a before log, focuses the VM, captures the current form, detects controls using OpenCV, fills the detected fields, clicks Add, and records the after log. If visual detection fails, it falls back to the older calibrated coordinate method. This makes Agent 1 more robust than a purely fixed-coordinate bot while still allowing recovery when vision fails.

Agent 2 uses explicit field clicks based on calibration, PyDirectInput key handling, and clipboard-based paste. This path is useful when the screen layout is stable and speed is more important than live detection.

Agent 3 writes patient data and calibrated coordinates into an INI payload and invokes an AutoHotkey script. This path exists because some VM and Win32 control combinations respond better to AutoHotkey than to Python-level input libraries.

3.4 Read Patient Details Workflow

The Read Patient Details workflow captures the patient details screen and extracts visible text. The bot loads calibration, navigates to the patient details area, focuses the VM, captures a timestamped screenshot, and passes the screenshot to the OCR pipeline. The extracted text is returned as words, combined text, screenshot path, timestamp, and word count.

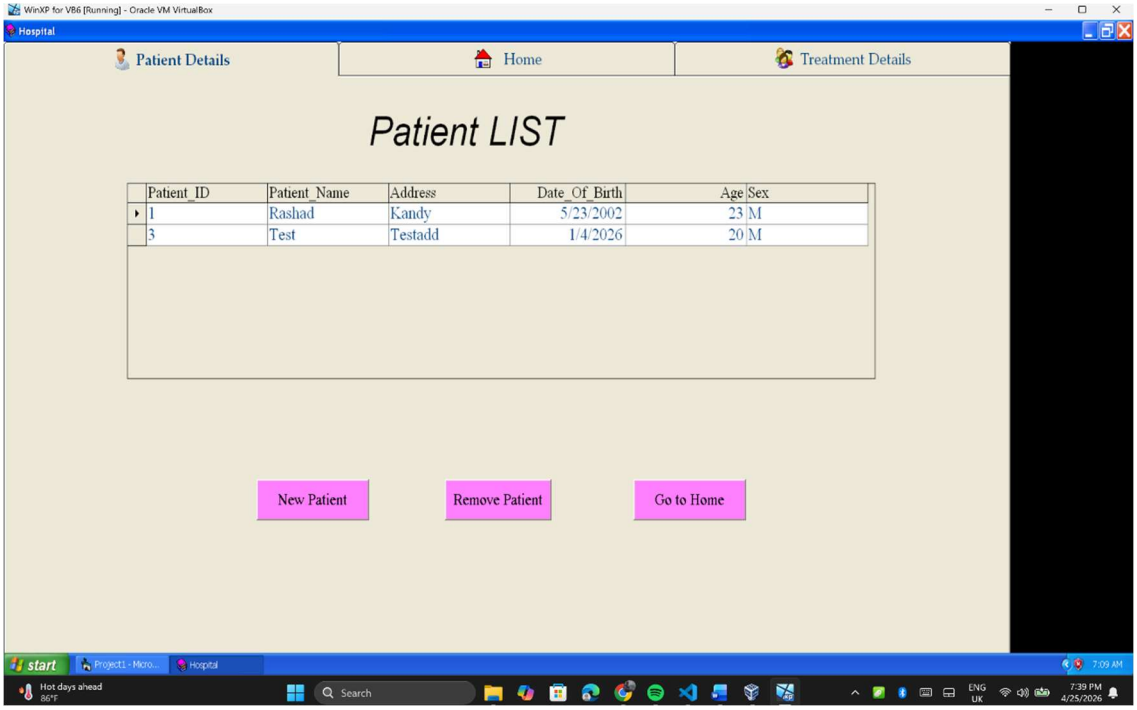


Figure 4: RPA read patient details

The OCR implementation is supported by table-oriented utilities that improve extraction from patient grids. These utilities apply preprocessing, table detection, column boundary detection, character whitelists, and field-specific cleanup. For example, patient IDs and ages are constrained to digits, dates are cleaned using date patterns, and sex values are normalized to "M" or "F".

This workflow is important because the bot layer must not only write data into the legacy system but also recover information from it. Readback is essential for verification, integration, and evidence generation.

3.5 Treatment Entry Workflow

The Treatment Entry workflow handles the legacy Fill Details dialog. The workflow includes patient selection, opening the treatment dialog, detecting the visible dialog area, identifying editable fields, entering treatment data, verifying that fields appear filled, and then clicking Add or Return depending on the selected action.

The treatment payload includes fields such as:

- Patient ID.

- Weight.
- Blood pressure.
- Pulse.
- Temperature.
- Medicine.
- Treatment taken.

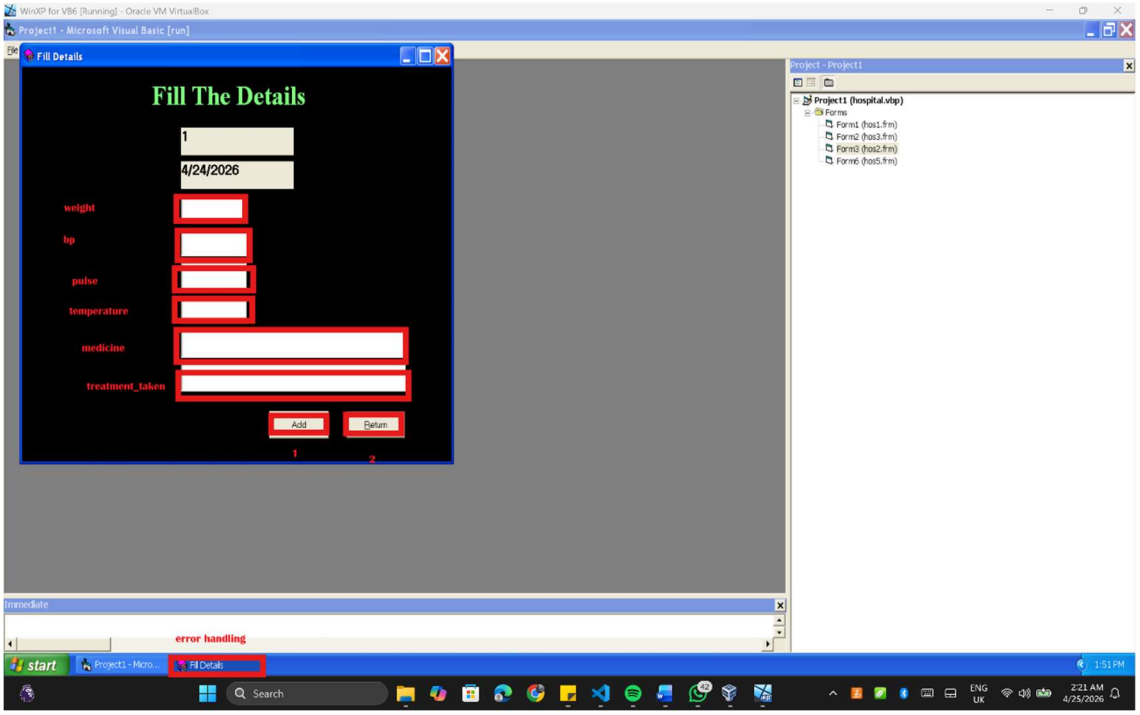


Figure 5: RPA add treatment details

The workflow includes special data sanitation because the Visual Basic 6 form is strict about some field formats. For example, slash-delimited blood pressure values can trigger a runtime type mismatch in the legacy application, so the live-form input is normalized to safer numeric content before entry.

Field entry uses AutoHotkey first where possible. If AutoHotkey is unavailable or fails, the bot falls back to PyAutoGUI and clipboard-based typing. After typing, the bot captures a before/after visual comparison of the field area and checks whether the number of dark pixels is consistent with text being present. If verification fails, it captures a debug screenshot and raises a field typing verification error.

The workflow also detects and dismisses Visual Basic runtime error dialogs. This is important because legacy applications often show modal popups that block further automation. Without popup handling, a single runtime dialog could cause the bot to continue clicking behind the modal window or hang indefinitely.

3.6 Delete/Treatment Workflow

The Delete/Treatment workflow automates deletion through the dedicated Delete Treatment dialog. It first ensures that the Treatment History screen is available, opens the required patient, clicks the delete button, waits for the Delete Treatment dialog, and then uses OCR and visual heuristics to identify dialog controls such as Delete and Return.

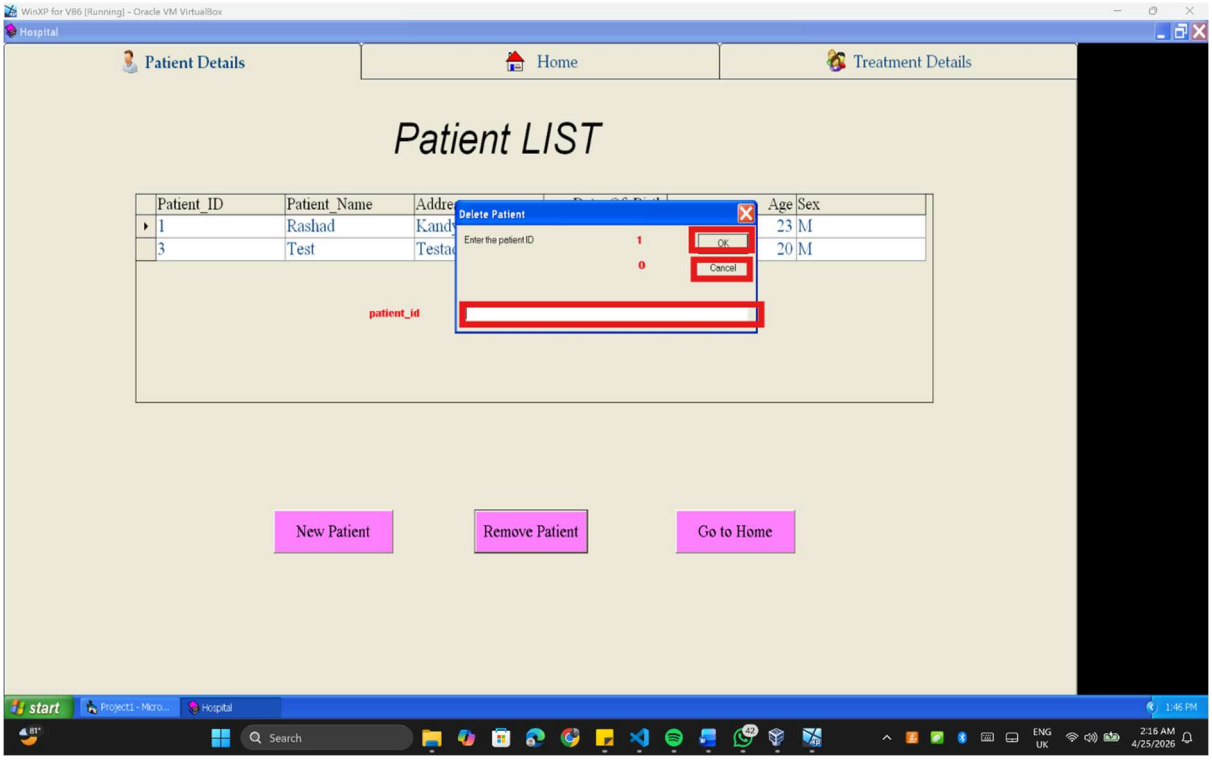


Figure 6: RPA remove patient

The workflow includes fallback geometry for button locations when OCR is weak. It also checks for runtime popups after deletion and attempts to dismiss them safely. After clicking Return, the bot verifies that it has returned to the Treatment History screen. This is a good example of screen-state gated execution: the workflow is not considered complete merely because a delete click occurred; it must return to an expected state.

3.7 Screen-State Recovery/ Error Handling

The screen-state recovery module supports workflows by routing the application back to known screens. If the bot needs the patient list but is on the dashboard, it clicks the calibrated Patient Details button. If it needs treatment history but is on the patient list, it clicks the calibrated Treatment Details button. If it is on the Add Patient form, it uses the detected Return button to leave the form.

This design reduces cascading failures. A bot may fail once because a previous workflow left the application on a different screen, but recovery allows the next workflow to start from a known state instead of assuming the screen is correct.

4. TESTING AND EVALUATION

4.1 Evaluation Goals

The evaluation focused on whether the RPA Bot Layer can execute key GUI workflows reliably and observably. The main evaluation questions were:

- Can the bot add a patient using only the GUI?
- Can the bot read patient details from the rendered screen?
- Can the bot enter treatment data into the Fill Details dialog?
- Can the bot execute Delete/Treatment operations and return to the expected screen?
- Can the bot remain robust when the VM is repositioned?
- Does each task produce enough evidence for debugging and audit?

4.2. Test Cases

The following test cases were derived from the proposal, the implemented workflows, and the component objectives:

1. When the Add Patient form is visible, the bot should detect or focus all required fields and submit the record.
2. When live Add Patient form detection fails, the bot should fall back to calibrated coordinate automation.
3. When the VM window is repositioned, relative-coordinate navigation should continue to click the correct logical controls.
4. When the bot is on the wrong screen, screen-state recovery should navigate to the required screen before execution.
5. When the patient details screen is captured, OCR should extract useful structured text from the rendered GUI.
6. When the Treatment Entry dialog is visible, the bot should detect fields, enter values, verify field fill, and handle runtime popups.
7. When the Delete Treatment dialog appears, the bot should identify Delete and Return controls and verify return to Treatment History.
8. For each task, before/after JSON logs and VM screenshots should be produced.

4.3 Metrics

The evaluation used the following metrics:

- Workflow success rate.
- Field-level input failure rate.
- Field location accuracy.
- OCR field-level accuracy.
- Average and median duration.
- Before/after JSON pairing rate.
- Before/after screenshot coverage.
- Error messages and failure causes.

The draft evaluation reported three key component metrics: field-level input failure rate of 1.9%, OpenCV field location accuracy of 96.7%, and OCR field-level accuracy of 94.1%. The local repository logs provide additional task-level evidence for the current implementation state.

4.4. Log-Based Evaluation

The current repository contains 28 scored runs in the before/after logging structure. The runs cover:

- `add_patient`: 6 runs.
- `add_treatment`: 8 runs.
- `delete_treatment`: 1 run.
- `local_no_api_demo`: 7 runs.
- `resolve_patient_id`: 6 runs.

For the direct bot-layer workflows, the strongest available result is the Add Patient workflow, which has six successful logged attempts out of six. Add Treatment is less stable, with five successes out of eight attempts. Delete Treatment has one successful logged attempt, but the sample size is too small to make a strong general claim.

The log analyzer also reports full observability for the direct operator-assisted task group: paired before/after JSON records, before screenshots, after screenshots, and structured after records are available for those runs. This confirms that the logging mechanism is functioning as intended.

4.5 Gantt Chart

The project timeline spans from July 2025 to June 2026 as illustrated in the Gantt chart below. Key phases include planning and requirement gathering (July–August 2025), system architecture design (August–September 2025), parallel development of all four core components (September–November 2025), system integration (November–December 2025), testing and evaluation (December 2025–January 2026), optimization and final report submission (February–April 2026), and defense and final viva (May–June 2026).

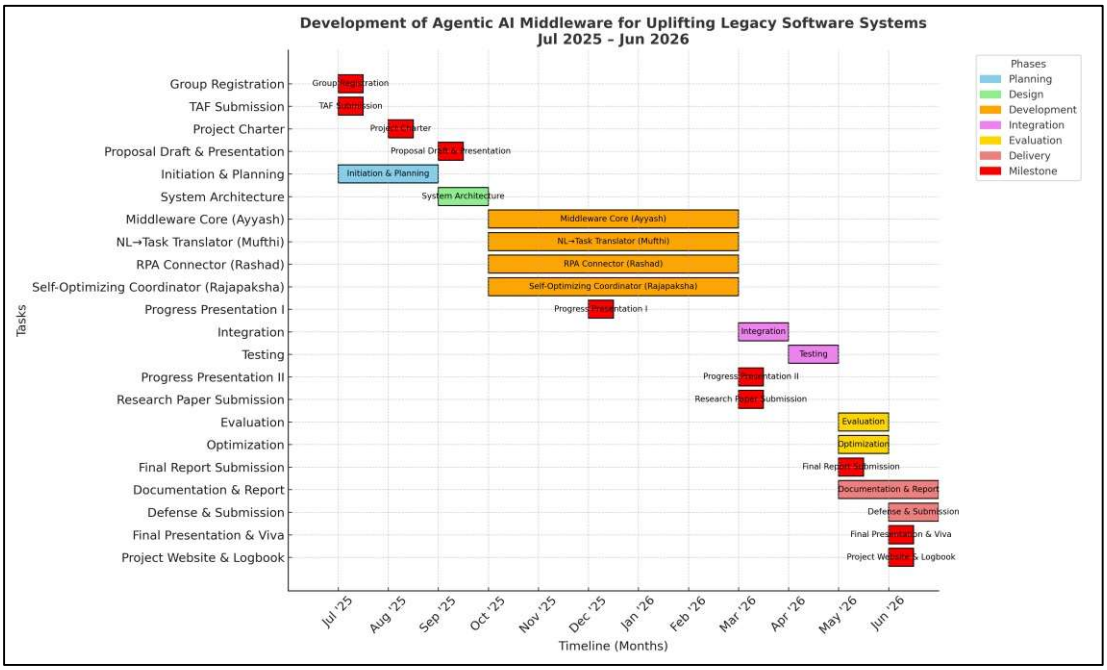


Figure 7: Gantt Chart

5. RESULTS AND DISCUSSION

5.1 Local Structured Log Results

Table 3: Local Structured Log Evaluation Results

Task	Runs	Successes	Failures	Success Rate	Average Duration (s)	Median Duration (s)
Add Patient	6	6	0	100.0%	55.59	51.04
Add Treatment	8	5	3	62.5%	18.85	15.31
Delete Treatment	1	1	0	100.0%	40.86	40.86

The Add Patient results show that the vision-first and fallback-based approach is effective for the tested patient registration workflow. The average duration of 55.59 seconds includes VM focus, screenshot capture, form filling, submission, and logging overhead. While this is slower than direct database insertion, it is acceptable for a non-intrusive legacy GUI bridge where safety and evidence are more important than raw speed.

The Add Treatment results are weaker. A 62.5% success rate shows that the treatment dialog is more difficult to automate than the Add Patient form. This is expected because it includes more fields, dialog restoration behavior, stricter legacy validation, field-level verification, and runtime error handling. The failures indicate that treatment entry needs further hardening before the workflow can be treated as production-ready.

The Delete Treatment result is positive but limited. A single successful run proves feasibility, but not reliability. More repeated trials are needed before drawing a strong conclusion.

5.2 Add Patient Agent Comparison

Table 4: Add Patient Agent Comparison

Add Patient Agent	Primary Strategy	Structured Runs	Successful Runs	Accuracy Proxy (%)	Mean Time (s)	Median Time (s)	Evaluation Status
Agent 1	Vision-first form detection with coordinate fallback and post-task logging	6	6	100.0	55.59	51.04	Fully benchmarked in current structured logs
Agent 2	Calibrated coordinate entry with PyDirectInput and clipboard-based typing	N/A	N/A	N/A	N/A	N/A	Implemented, but not benchmarked in current structured logs
Agent 3	AutoHotkey runner using calibration-driven INI payload injection	N/A	N/A	N/A	N/A	N/A	Implemented, but not benchmarked in current structured logs

This comparison shows an important evaluation gap. The implementation includes three execution paths, but only Agent 1 currently has structured benchmark evidence. Agent 2 and Agent 3 should be tested under the same logging harness before any claim is made about which path is fastest or most reliable.

5.3. Component Accuracy Results

The draft evaluation reports the following component-level results:

- Field-level input failure was reduced to 1.9% through the three-strategy hybrid approach.
- OpenCV-based field location accuracy reached 96.7% across evaluated screens.
- Relative-coordinate calibration remained stable under deliberate VM repositioning tests.
- OCR field-level accuracy reached 94.1%, with most difficulty occurring in long or visually ambiguous table entries.

These results support the central design decision: reliability improves when the bot does not depend on only one method. Relative calibration helps with VM movement. OpenCV helps with screen variation. PyDirectInput and AutoHotkey help when PyAutoGUI input is not captured reliably. OCR provides readback when the legacy system does not expose data programmatically.

5.4 Discussion

The implemented RPA Bot Layer demonstrates that practical legacy modernization can be achieved without replacing the legacy application immediately. The bot layer does not pretend that the old system is modern. Instead, it treats the GUI as the available integration boundary and strengthens that boundary with calibration, visual perception, fallback input, OCR, and logs.

The most successful part of the implementation is Add Patient automation. The workflow is well-scoped, has predictable fields, and benefits from live OpenCV detection. The fallback path also reduces the risk of total failure when visual detection is imperfect. This makes Add Patient a strong demonstration of the component.

Treatment Entry is a more complex workflow. It reveals the real difficulty of automating legacy Win32 applications: dialogs may appear behind the main window, input formats may be stricter than expected, runtime errors may appear, and field verification may be imperfect. The treatment workflow is therefore valuable not only because it automates a task, but because it exposes the kinds of reliability engineering required for legacy modernization.

The Delete/Treatment workflow demonstrates the importance of state-based automation. Deletion is a sensitive action, so it should not be executed blindly. The workflow waits for the dialog, identifies controls, performs the delete operation, handles possible runtime popups, and checks that the system returns to Treatment History. This pattern is appropriate for high-risk GUI automation.

The local logs also show that the broader end-to-end local demo flow is not yet as stable as the individual direct workflows. The `local_no_api_demo` attempts failed in the available scored logs, and `resolve_patient_id` succeeded in two of six runs. These failures do not invalidate the RPA Bot Layer, but they show that orchestration and patient-ID resolution need further integration work before the complete no-API workflow can be considered reliable.

6. LIMITATIONS

The main limitations are as follows:

1. The benchmark sample size is small. Add Patient has six structured runs, Add Treatment has eight, and Delete Treatment has only one. More trials are required for statistically stronger conclusions.
2. Agent 2 and Agent 3 are implemented but not benchmarked under the same structured logging harness as Agent 1.
3. Some add-patient fallback paths still use pixel offsets rather than fully normalized relative coordinates.
4. Treatment Entry remains less stable than Add Patient because it involves modal dialog behavior, stricter field formats, and runtime error handling.
5. OCR is useful but not perfect. Long addresses, visually crowded tables, and low-quality screenshots can reduce extraction accuracy.
6. The implementation is Windows-centric because PyDirectInput and AutoHotkey are specific to Windows automation.
7. The current bot layer uses practical OpenCV heuristics rather than a fully trained semantic UI model. This is appropriate for the prototype but may require retraining or reconfiguration for very different legacy screens.
8. The broader REST/GraphQL and dynamic simulator vision from the proposal is not fully evaluated as part of this individual bot-layer report.
9. End-to-end orchestration logs show that patient-ID resolution and complete no-API demo flow still require stabilization.

7. FUTURE WORK

Future work should focus on strengthening reliability, configurability, and integration.

First, the hybrid input methods should be consolidated under a runtime strategy manager. PyAutoGUI can remain the default interaction method, PyDirectInput can be selected automatically when keystrokes are dropped, and AutoHotkey can be used as the final fallback for difficult Win32 controls. The selected strategy, fallback count, error reason, and per-field latency should be logged for every sensitive action.

Second, all workflow paths should be standardized under the relative-coordinate calibration model. Add Patient Agent 2 and Agent 3 should avoid legacy pixel-only calibration where possible. A single calibration format would improve maintainability and reduce confusion across workflows.

Third, field-level verification should be expanded. After entering sensitive values, the bot should verify the field using OCR, pixel-density checks, or value-specific visual assertions. When verification fails, the bot should follow a deterministic recovery sequence: retry, escalate input strategy, and re-navigate to a known screen.

Fourth, perception and interaction parameters should be externalized into configuration files. Thresholds for contour filtering, OCR character whitelists, render delays, retry limits, and dialog detection should be adjustable without changing workflow code.

Fifth, Agent 2 and Agent 3 should be benchmarked using the same before/after logging framework as Agent 1. This would allow a fair comparison of speed, reliability, and failure patterns.

Sixth, a bot registry should be introduced so workflow-specific handlers can be activated, disabled, replaced, or versioned. This would support gradual extension to more hospital screens without creating tightly coupled scripts.

Seventh, the proposal's dynamic GUI simulator idea should be revisited as a digital-twin testing environment. Recorded VM states and screenshots could be replayed against bot scripts to support regression testing without always using the live legacy application.

Finally, the broader architecture should connect the hardened bot layer to secure API endpoints with governance features such as authentication, RBAC, audit logging, PII masking, and approval workflows. These features are especially important in healthcare contexts where patient data must be protected.

8. CONCLUSION

5

This report presented the design, implementation, and evaluation of an RPA Bot Layer for legacy system modernization. The component addresses a practical modernization problem: how to automate a GUI-only Visual Basic 6 hospital system running inside a Windows XP virtual machine without modifying the legacy application.

The implemented solution combines relative-coordinate calibration, OpenCV-based visual detection, PyAutoGUI, PyDirectInput, AutoHotkey, Tesseract-supported OCR pipelines, screen-state detection, and structured before/after JSON and screenshot logs. It supports Add Patient, Read Patient Details, Treatment Entry, and Delete/Treatment workflows. The strongest current result is the Add Patient workflow, which achieved six successful runs out of six in local structured logs. The treatment workflows demonstrate feasibility but require further hardening.

The main contribution of this component is not simply that it automates a form. It shows how a legacy GUI can be treated as a reliable integration boundary when automation is supported by perception, calibration, fallback strategies, state checks, and evidence logging. This makes the RPA Bot Layer a practical foundation for broader low-code legacy modernization and for future integration with agentic orchestration, simulators, and secure APIs.

REFERENCES

[1] M. Bisbal, D. Lawless, B. Wu, and J. Grundy, "Legacy information systems: Issues and directions," IEEE Software, vol. 16, no. 5, pp. 103-111, Sep.-Oct. 1999, doi: 10.1109/52.795103.

[2] K. H. Bennett, "Legacy systems: Coping with success," IEEE Software, vol. 12, no. 1, pp. 19-23, Jan. 1995, doi: 10.1109/52.363157.

[3] R. N. Charette, "Inside the hidden world of legacy IT systems," IEEE Spectrum, Aug. 2020. [Online]. Available: https://spectrum.ieee.org/inside-hidden-world-legacy-it-systems

[4] S. Aguirre and A. Rodriguez, "Automation of a business process using robotic process automation (RPA): A case study," in Applied Computer Sciences in Engineering, Cham, Switzerland: Springer, 2017, pp. 65-71, doi: 10.1007/978-3-319-66963-2_7.

[5] M. C. Lacity and L. P. Willcocks, Robotic Process Automation and Risk Mitigation: The Definitive Guide. Stratford-upon-Avon, U.K.: SB Publishing, 2017.

[6] R. Oberhauser and A. Pogolski, "Adaptive robotic process automation," in Proc. IEEE Int. Conf. Autonomic and Intelligent Systems (AIMS), 2019, pp. 29-36.

[7] T. Beltramelli, "pix2code: Generating code from a graphical user interface screenshot," in Proc. ACM SIGCHI Symp. Engineering Interactive Computing Systems (EICS), 2018, pp. 1-6.

[8] B. Li et al., "RICO: A mobile app dataset for building data-driven design applications," in Proc. 30th Annu. ACM Symp. User Interface Software and Technology (UIST), 2017, pp. 845-854, doi: 10.1145/3126594.3126651.

[9] J. Luo et al., "OmniParser for pure vision based GUI agent," arXiv preprint arXiv:2408.00203, 2024.

[10] C. Wang et al., "UI-TARS: Pioneering automated GUI interaction with native agents," arXiv preprint arXiv:2501.12326, 2025.

[11] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629-633, doi: 10.1109/ICDAR.2007.4376991.

[12] A. M. Memon, "An event-flow model of GUI-based applications for testing," Software Testing, Verification and Reliability, vol. 17, no. 3, pp. 137-157, Sep. 2007.

[13] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2023.

[14] G. Wang et al., "Voyager: An open-ended embodied agent with large language models," Trans. Machine Learning Research, 2024.

[15] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[16] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[17] L. Byron, N. Schrock, and D. Hartig, "GraphQL: A data query language," 2015. [Online]. Available: https://graphql.org/learn/

APPENDICES

Appendix A: Suggested Figures for Final Word Submission

Fig. 1 should show the broader architecture with the RPA Bot Layer between modern orchestration services and the Windows XP Visual Basic 6 hospital application.

Fig. 2 should show the execution pipeline: request payload, VM focus, screen-state detection, visual detection/calibration, input execution, verification, and before/after logging.

Fig. 3 should illustrate relative-coordinate calibration using VM bounds and normalized `r_x`, `r_y` points.

Fig. 4 should show an Add Patient screenshot with detected input fields and Add/Return buttons.

Fig. 5 should show sample before/after JSON records and screenshots from `rpa_logs/Before` and `rpa_logs/After`.

Fig. 6 should show the Treatment Entry Fill Details dialog detection and field verification process.

Fig. 7 should show the Delete/Treatment workflow from Treatment History to Delete Treatment dialog and back.

Appendix B: Implementation Evidence

Key local implementation files include:

- `pages/vm_window.py`: VM discovery, focusing, maximizing, and screenshot region calculation.
- `pages/add_patient_vision.py`: OpenCV-based Add Patient form control detection and filling.
- `pages/patient_details/new_patient/agent1/automator.py`: Add Patient Agent 1 with task logging and fallback.
- `pages/new_patient/agent2/automator.py`: coordinate-driven PyDirectInput and clipboard entry.
- `pages/new_patient/agent3/agent3_runner.py`: AutoHotkey payload runner.
- `pages/screen_state.py`: dashboard, patient list, treatment history, add-patient form, and unknown-state detection.
- `pages/patient_details/read_patient_details.py`: screenshot capture and OCR-supported patient details extraction.
- `read/main_improved.py`: Tesseract-based columnar OCR utilities.
- `pages/treatment/add_treatment/add_treatment_vision.py`: Fill Details dialog detection, typing, verification, and runtime error handling.
- `pages/treatment/delete_treatment_dialog.py`: Delete Treatment dialog OCR/button detection and recovery.
- `pages/rpa_task_logger.py`: before/after JSON and screenshot logging.
- `analyze_rpa_logs.py`: log pairing, duration calculation, success-rate scoring, and observability analysis.

Appendix C: Commercialization and Deployment Outlook

The proposal described a three-phase commercialization path. For this individual component, the path can be adapted as follows:

Phase 1 should deploy workflow-specific bots in a controlled pilot environment for selected legacy screens. The goal is to collect failure cases, tune calibration, and harden the workflows.

Phase 2 should package the bot layer as a configurable low-code connector for organizations that need non-intrusive legacy automation without full replacement.

Phase 3 should integrate the hardened bot layer into a larger modernization platform with orchestration, APIs, monitoring, governance, and support services.

Appendix D: Evidence-Based Limitations Summary

Table 5: Limitations and Future Improvements

Area	Current Status	Required Improvement
Add Patient Agent 1	Six successful structured runs	Increase sample size and test under more VM positions
Add Patient Agent 2	Implemented	Add structured benchmark logging
Add Patient Agent 3	Implemented	Add structured benchmark logging
Add Treatment	Five successes out of eight logged runs	Improve field verification and dialog recovery
Delete Treatment	One successful logged run	Run more repeated trials
Patient ID resolution	Two successes out of six logged runs	Improve OCR/search/readback pipeline
End-to-end no-API demo	Failed in available local scored logs	Stabilize orchestration and recovery between tasks
OCR	Useful but imperfect	Improve preprocessing, validation, and human-review safeguards

Area	Current Status	Required Improvement
Calibration	Relative coordinates used in main navigation/treatment flows	Standardize all agents under relative calibration